Container truncation

Document #	D3526R0
Date	2024-11-22
Project	Programming Language C++
Targeted subgroups	LEWG
Target release	C++26
Reply-to	Peter Bindels <dascandy@gmail.com>

§ 1 Abstract

std::vector (et al) have a function to change the number of stored allocations, called `resize`. This may need to enlarge the container, which is an operation that cannot generally be done noexcept. Shrinking a container, however, can always be done noexcept, as it only destructs member objects using their noexcept-by-default destructor, and reducing the stored size to a lower number. This has benefits for code generation, where the compiler will be able to know that the truncation cannot throw without further analysis.

It enables truncating containers with types that are not default-constructible. Right now, if you have a container with non-default-constructible types, there is no way to truncate, other than to repeatedly pop_back(). The resulting compile error is not immediately obvious, as the mental model of the user is "I am only removing items since I truncate the container".

In addition, it helps code readability to be able to specify that a given code fragment will only reduce the size of a buffer, which in some contexts is a common operation.

§ 2 Wording

In the summaries of `vector`, `inplace_vector`, `list`, `forward_list`, `deque` and `vector<bool>`, add the function

```
    constexpr void      truncate(size_type sz) noexcept;
```

In the corresponding explanation blocks following, add the following

```
void truncate(size_type sz) noexcept;
```

```
expects: T is MoveInsertable into `deque`

Precondition: `sz` is less than or equal to `size()`

effects: Erases the last `size() - sz` elements from the sequence.
```